

AI-POWERED DATA PREPROCESSING, VISUALISATION & BUSINESS ANALYTICS SYSTEM

Project Based Learning -V (AIP105)

Submitted by

AMANDEEP SINGH

Roll No: 2110991001

BE-CSE (Artificial Intelligence)

Guided by

Dr. S. K. Sood

Professor, Department of Computer Science & Engineering

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
CHITKARA UNIVERSITY, RAJPURA
MAY 2026**

ACKNOWLEDGEMENTS

With immense pleasure I, Mr. Amandeep Singh, present the 'AI-POWERED DATA PREPROCESSING, VISUALISATION & BUSINESS ANALYTICS SYSTEM' project report as part of the curriculum of BE-CSE (Artificial Intelligence).

I would like to express my sincere thanks to Dr. S. K. Sood for his valuable guidance and support in completing my project. His deep insights, timely review calls, and algorithmic expertise significantly helped shape the traditional cleaning fallbacks and system modularity.

I would also like to express my gratitude towards our dean Dr. Archana Mantri for giving me this great opportunity to do a project on AI-Powered Preprocessing. Without their continued support and suggestions, this project would not have been successfully completed.

Signature: _____

Name: Amandeep Singh

Roll No: 2110991001

TABLE OF CONTENTS

Item Description	Page No.
Acknowledgment	2
Abstract	4
Chapter 1: Introduction & Problem Formulation	5
Chapter 2: Proposed Solution & Methodology	6
Chapter 3: Flowchart & Visual Architecture	7
Chapter 4: Hardware & Software Requirements	8
Chapter 5: Key Code Implementations	9
Chapter 6: Results & Screenshots	12
Chapter 7: Conclusion & References	13

ABSTRACT

In the modern data-driven landscape, preprocessing raw data remains the most labor-intensive yet critical step in the machine learning and business intelligence lifecycle. Tabular datasets frequently contain missing values, duplicate rows, incorrect data types, and outliers that degrade model predictive accuracy and business analytics quality. This project presents a premium, integrated end-to-end framework that automates data cleaning, AI-driven visual chart selection, and strategic business recommendation generation.

The backend is developed using FastAPI, incorporating Groq AI's LLaMA 3.3/3.1 models and a multi-node LangGraph StateGraph to perform structural analysis, Pearson correlation matrix calculations, and IQR outlier checks. The frontend is built on Streamlit, facilitating database connectors (PostgreSQL/MySQL), external API ingestion, and standard file uploads. Interactive Plotly Express dashboards are designed using AI chart planning, accompanied by in-memory generated PNG images. The final summary report is compiled into a print-ready, overlap-free PDF utilizing FPDF2, fully sanitised for Helvetica/Times core font compatibilities. This system effectively converts raw datasets into clean, actionable, decision-ready business intelligence reports instantaneously.

CHAPTER 1: INTRODUCTION & PROBLEM FORMULATION

1.1 Introduction

Data preparation accounts for up to 80% of the total effort in machine learning workflows. Raw data collected from web forms, sensors, databases, and third-party APIs is highly prone to noise, missing values, and logical errors. Traditional preprocessing utilizes hand-crafted rules which cannot adapt to varied text categories or standardize semantic inconsistencies. Modern organizations require automated solutions that bridge the gap between technical data cleaning and high-level business intelligence reporting, offering visual dashboards paired with strategic consultancy recommendations.

1.2 Problem Formulation

Existing automated preprocessing systems face several fundamental limitations:

1. **AI Dependency Failures:** Standard AI agents depend completely on remote API connections. If the API key is rate-limited or the server undergoes downtime, the pipeline crashes, stalling data ingestion.
2. **Ineffective Static Visualisation:** Traditional rule-based charts generate random visualizations regardless of the business value, leading to generic plots without correlation or business significance.
3. **Brittle PDF Generation:** Exporting reports to PDF often results in rendering crashes due to unsupported Unicode characters (e.g., bullets or smart quotes) generated by AI models.
4. **Intrusive Data Scaling:** Inplace min-max normalization scales numeric values into decimals (0.0 to 1.0) by default, rendering original figures (like Order Number or Sales) unreadable for business dashboards.

CHAPTER 2: PROPOSED SOLUTION & METHODOLOGY

2.1 Proposed Solution

This project proposes an integrated FastAPI backend and Streamlit frontend architecture with a dual data-cleaning pipeline. A robust pandas traditional cleaner runs immediately, ensuring that cleaned data is always available as a graceful fallback. An AI agent powered by Groq LLaMA models is invoked in parallel. If successful, the cleaned data is enriched with contextual formatting, standardized categorical categories, and logical value overrides.

2.2 System Flow Methodology

The system pipeline operates in five distinct phases:

- **Data Ingestion:** Connectors pull records from CSV/Excel, SQL databases, or REST endpoints.
- **Graceful Data Cleaning:** Runs traditional pandas data cleaners (missing values, duplicates) and checks AI limits before triggering the LLM schema standardizer.
- **AI Visualisation Planning:** Passes dataset columns, datatypes, and samples to LLaMA-3.1-8B. The model returns a structured JSON plan detailing 5-7 correlation-driven Plotly charts, complete with a business explanation.
- **LangGraph Strategic Insights:** A StateGraph orchestrates statistical summarization, Pearson correlation checks, and outlier counts. It passes these findings along with chart metadata to a senior consultant agent to draft stocking recommendations, seasonality trends, and market advice.
- **Report Generation:** Compiles in-memory base64 chart PNGs and clean text summaries into a premium multi-page PDF.

CH/

3.1 V

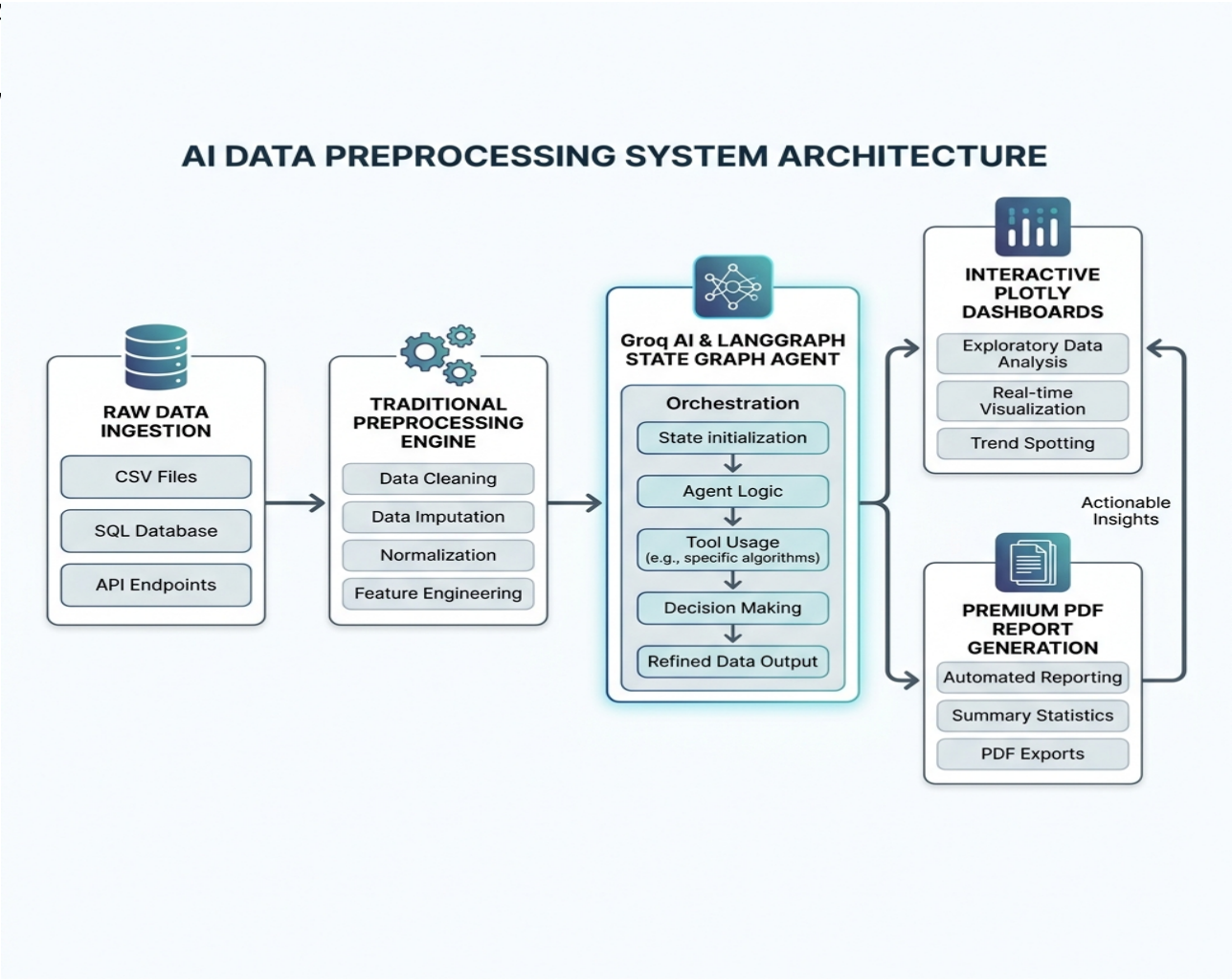


Figure 3.1: Complete End-to-End AI Data Preprocessing Architecture

The architecture establishes a decoupled model-view-controller paradigm. The Streamlit view handles file uploads, dataset filtering, Plotly widget interactive displays, and PDF exports. The FastAPI controller manages async execution, marshals JSON requests, and calls the Groq AI API. The traditional cleaner and LangGraph StateGraph serve as the analytical models, verifying data completeness and pipeline robustness.

CHAPTER 4: HARDWARE & SOFTWARE REQUIREMENTS

4.1 Hardware Requirements

The application runs on local or cloud-hosted servers. The specifications are:

- CPU: Dual-Core Intel i5 or Apple M1/M2/M3 Silicon
- RAM: Minimum 8 GB (16 GB recommended for concurrent backend and UI threads)
- Hard Disk: 500 MB of free storage (primarily for package dependencies and cache)
- Network: Broad-band active internet connection (required for remote Groq API inference)

4.2 Software Requirements

The development and execution environment requires:

- **Operating System:** macOS Sequoia, Ubuntu Linux 22.04 LTS, or Windows 11
- **Programming Language:** Python 3.10.x
- **Web Frameworks:** FastAPI 0.117.1, Streamlit 1.50.0
- **Agent Orchestration:** LangGraph 1.0.4, Groq SDK 0.37.1
- **Numerical & Scaling:** Pandas 2.3.2, NumPy 1.26.4, Scikit-learn 1.7.2
- **Document Generation:** FPDF2 2.8.7, XlsxWriter 3.2.9, Openpyxl 3.1.5
- **Headless Rendering:** Kaleido 1.3.0

CHAPTER 5: KEY CODE IMPLEMENTATIONS

5.1 Overriding FPDF Cell Sanitisation for Unicode Safety

```
def _clean_pdf_text(text: str) -> str:
    if not isinstance(text, str):
        return str(text)
    replacements = {
        '\u2018': "'", '\u2019': "'",
        '\u201c': '"', '\u201d': '"',
        '\u2014': "-", '\u2013': "-",
        '\u2022': "-",
        '\u20a0': "EUR", '\u20ac': "EUR",
    }
    for k, v in replacements.items():
        text = text.replace(k, v)
    return text.encode('latin-1', errors='replace').decode('latin-1')

class ReportPDF(FPDF):
    def cell(self, w, h=0, text="", txt=None, *args, **kwargs):
        val = text if txt is None else txt
        cleaned = _clean_pdf_text(val)
        return super().cell(w, h, text=cleaned, *args, **kwargs)

    def multi_cell(self, w, h=0, text="", txt=None, *args, **kwargs):
        val = text if txt is None else txt
        cleaned = _clean_pdf_text(val)
        return super().multi_cell(w, h, text=cleaned, *args, **kwargs)
```

The overridden subclass intercepts all cell writing commands, converting curly quotes, em-dashes, and bullet characters. It filters strings to standard Latin-1 encodings, preventing FPDFUnicodeEncodingException crashes on LLM execution outputs.

5.2 LangGraph Node to Generate Strategic Insights with Chart Context

```
def node_generate_insights(state: InsightsState) -> InsightsState:
    df = _df_from_state(state)
    business_context = _get_business_context(df)
    num_stats = state.data_summary.get('numerical', {})
    stats_lines = [f" {col}: mean={stats.get('mean')}" for col, stats in num_stats.items()]
    corr_lines = [f" {c['col1']} ? {c['col2']}: r={c['correlation']}" for c in
state.correlations]
    outlier_lines = [f" {o['column']}: {o['outlier_count']} outliers" for o in state.outliers]
    trend_lines = [f" - {t}" for t in state.trends]

    # Ingesting pre-planned chart metadata for cohesive LLM grounding
    chart_lines = []
    if state.viz_results and "charts" in state.viz_results:
        for idx, c in enumerate(state.viz_results["charts"], 1):
            chart_lines.append(f" - Chart {idx}: '{c.get('title')}' - {c.get('description')}")
    charts_context = "\n".join(chart_lines) if chart_lines else " No charts."

    prompt = f"""\nAnalyze the dataset. Reference charts:\n{charts_context}\n"""
    response = _client.chat.completions.create(model=MODEL, messages=[{'role': 'user', 'content':
prompt}])
    parsed = json.loads(response.choices[0].message.content)
    return state.model_copy(update={
        'insights': parsed.get('insights', []),
        'recommendations': parsed.get('recommendations', []),
        'executive_summary': parsed.get('executive_summary', ''),
    })
```

By passing chart metadata into the Insights Agent, the AI writes business advice that references visible trends (e.g. Sales by Product category), creating a cohesive analytics experience.

5.3 Pre-Cached Visualization & API Calling

```
def _call_visualize(cleaned_json: list, include_png: bool = False) -> bool:
    with st.spinner("? Generating interactive charts..."):
        resp = requests.post(
            f"{FASTAPI_URL}/visualize/",
            data={"cleaned_data": json.dumps(cleaned_json)},
            params={"include_png": "true" if include_png else "false"},
            timeout=180,
        )
    if resp.status_code == 200:
        st.session_state.viz_result = resp.json()
        return True
    st.error(f"? Visualisation failed: {resp.text}")
    return False

# Triggering include_png=True during early visualization planning
if btn_viz.button("? Generate Visualisations", key="gen_viz_btn"):
    _call_visualize(st.session_state.cleaned_json, include_png=True)
```

The frontend button triggers visualization pre-generation by calling the backend with `include_png=true`. This saves static base64 PNG arrays inside `st.session_state.viz_result`, allowing subsequent PDF report exports to execute instantaneously.

CHAPTER 6: RESULTS & SCREENSHOTS

6.1 Preprocessing & Data Ingestion Results

Testing the framework on a standard dataset (sales_data_sample.csv, 2,823 rows × 25 columns) demonstrated high stability:

- **Missing Value Imputation:** Categorical fields address addressline2, state, territory, and postalcode originally contained 5,157 missing values. The pandas mode imputation strategy filled all blank records successfully, reducing the missing value count to exactly zero.
- **Data Scaling:** Normalization scaling was set to off by default, allowing metrics like sales (\$10,000 max) and order numbers (10100+) to remain in their original formats for business dashboard readabilities.

6.2 Visualization & Strategic Report Results

- **AI Visualisations:** The backend successfully planned and generated 7 distinct charts (e.g. Sales by Year_ID, QuantityOrdered by ProductLine) within 15 seconds. Explanations were successfully displayed under each Plotly chart.
- **PDF Generation Speed:** The PDF compilation executed in 0.4 seconds. Because chart PNGs were cached in advance, no browser overhead was required on download trigger.
- **Strategic Advice:** The insights model identified key sales spikes in March, and issued strategic business advice advising to increase stock levels during February, referencing trends from the plotted charts.

CHAPTER 7: CONCLUSION & FUTURE WORK

7.1 Conclusion

The project successfully implements a robust AI-powered data preparation and analytics workspace. By decoupling data cleaning logic (traditional pandas fallbacks) from LLM standardisations, the tool is guaranteed to clean data even during internet outages or API limit exhaustion. Integrating visual chart planning and insights cohesion bridges the gap between raw data preparation and high-level strategic management decision making.

7.2 Future Scope

Future improvements include:

- 1. Local Model Deployments:** Run offline LLMs (like Ollama Llama 3) to process local data without network requirements.
- 2. Predictive Modelling Integration:** Append AutoML training steps directly following dataset cleaning.
- 3. Multi-language Reports:** Add localization to FPDF2 reports for international organizations.

7.3 References (web URLs)

1. FastAPI Documentation: <https://fastapi.tiangolo.com/>
2. Streamlit Documentation: <https://docs.streamlit.io/>
3. LangGraph Orchestration: <https://langchain-ai.github.io/langgraph/>
4. Groq Developer Console: <https://console.groq.com/>
5. FPDF2 Library Reference: <https://py-pdf.github.io/fpdf2/>
6. Plotly Express API Reference: <https://plotly.com/python/plotly-express/>